

Tarefa 7: OMP Tasks com Lista Encadeada

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Objetivo

Este experimento tem como objetivo criar uma lista encadeada de nomes de arquivo fictícios e processá-la em paralelo utilizando a diretiva `#pragma omp task` do OpenMP. São implementadas e comparadas duas versões: uma ingênua (problemática) e uma correta, evidenciando os efeitos do uso de `#pragma omp single` e `firstprivate` na criação de tarefas a partir de estruturas de dados encadeadas.

2. Ambiente de Execução

Parâmetro	Valor
CPU	AMD Ryzen 5 5600G
Núcleos físicos	4
Compilador	GCC com -O2 -fopenmp
Nós na lista	10
Threads	4

3. Implementação

3.1 Estrutura da Lista

A lista encadeada é composta por nós do seguinte tipo:

```
typedef struct no { const char *nome; struct no *next; } no_t;
```

Foram criados 10 nós com os seguintes nomes fictícios de arquivo: `relatorio_jan.txt`, `vendas_q1.csv`, `backup_2024.tar.gz`, `log_sistema.log`, `config.yaml`, `notas_reuniao.docx`, `planilha_rh.xlsx`, `imagem_disk.iso`, `script_deploy.sh` e `dados_clientes.db`.

3.2 Versão 1 — Problemática

Nesta versão, todas as threads do bloco parallel percorrem a lista de forma independente, cada uma criando uma tarefa para cada nó encontrado:

```
#pragma omp parallel num_threads(4)
{
    no_t *p = head;
    while (p) {
        #pragma omp task firstprivate(p)
        processar(p);
        p = p->next;
    }
    #pragma omp taskwait
}
```

Como cada uma das 4 threads percorre os 10 nós independentemente, são criadas $4 \times 10 = 40$ tarefas no total, resultando em cada arquivo sendo processado 4 vezes.

3.3 Versão 2 — Correta

A versão correta restringe a criação de tarefas a uma única thread com `#pragma omp single`, enquanto as demais ficam disponíveis para executar as tarefas geradas:

```
#pragma omp parallel num_threads(4)
{
    #pragma omp single
    {
        no_t *p = head;
        while (p) {
            #pragma omp task firstprivate(p)
            processar(p);
            p = p->next;
        }
    } /* taskwait implícito */
}
```

Apenas uma thread percorre a lista e cria 10 tarefas. As 4 threads competem para executá-las, e cada arquivo é processado exatamente uma vez.

4. Resultados

4.1 Versão Problemática

Nas três execuções realizadas, foram produzidas sempre 40 linhas de saída — confirmando a duplicação de trabalho. A tabela abaixo sumariza os resultados:

Execução	Linhas produzidas	Arquivos únicos processados	Duplicatas por arquivo
1	40	10	4×
2	40	10	4×
3	40	10	4×

Embora a contagem de linhas seja determinística (sempre 40), a distribuição das tarefas entre as threads e a ordem de impressão variam a cada execução, pois o escalonador de tarefas do OpenMP não é determinístico.

4.2 Versão Correta

Nas três execuções, foram produzidas exatamente 10 linhas. Cada arquivo apareceu precisamente uma vez. A tabela abaixo ilustra uma das execuções:

Thread	Arquivo Processado
3	relatorio_jan.txt
2	log_sistema.log
1	vendas_q1.csv
1	notas_reuniao.docx
3	planilha_rh.xlsx
2	script_deploy.sh
2	dados_clientes.db
0	backup_2024.tar.gz
1	imagem_disk.iso
2	config.yaml

5. Análise

5.1 Todos os nós foram processados?

Em ambas as versões, todos os 10 nós da lista foram processados. Na versão problemática, nenhum nó foi ignorado pois todas as 4 threads percorreram a lista até o final. Na versão correta, o único thread que percorre a lista garante a cobertura completa.

5.2 Algum nó foi processado mais de uma vez?

Na versão problemática, todos os nós foram processados 4 vezes, resultando em 40 execuções da função processar(). A contagem total é determinística: $nthreads \times nnós = 4 \times 10 = 40$. Na versão correta, nenhum nó foi duplicado.

5.3 O comportamento muda entre execuções?

Na versão problemática: a contagem de linhas (40) é estável, mas a ordem de saída e a alocação de tarefas por thread variam por não-determinismo do escalonador. Na versão correta: o conjunto de arquivos processados é invariável, porém a ordem das impressões varia pois as 4 threads competem pelas 10 tarefas disponíveis.

5.4 Como garantir que cada nó seja processado uma única vez?

A solução envolve três mecanismos combinados:

Mecanismo	Papel
<code>#pragma omp single</code>	Restringe a criação de tarefas a uma única thread, eliminando duplicação.
<code>firstprivate(p)</code>	Captura o valor corrente do ponteiro p no momento da criação da tarefa, evitando que a tarefa receba um ponteiro já avançado.
taskwait implícito	Ao sair do bloco single, o OpenMP aguarda todas as tarefas criadas internamente, garantindo que nenhum nó seja ignorado.

6. Conclusão

O experimento demonstrou de forma clara os riscos da criação indiscriminada de tarefas OpenMP em regiões paralelas sem controle explícito. A versão ingênua produziu 4× mais trabalho do que o necessário, violando a semântica esperada da aplicação. A correção foi simples: adicionar `#pragma omp single` para que apenas uma thread percorra a lista e crie as tarefas, enquanto as demais as consomem — esse é o padrão correto para paralelizar travessias de estruturas sequenciais com `omp task`.

A combinação de `single`, `firstprivate` e `taskwait` implícito é o tripé que garante correção, eficiência e ausência de condições de corrida ao paralelizar listas encadeadas com o modelo de tarefas do OpenMP.

7. Código Fonte

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define NTHREADS 4
6
7  typedef struct no { const char *nome; struct no *next; } no_t;
8
9  static const char *nomes[] = {
10     "relatorio_jan.txt", "vendas_q1.csv", "backup_2024.tar.gz",
11     "log_sistema.log", "config.yaml", "notas_reuniao.docx",
12     "planilha_rh.xlsx", "imagem_disk.iso", "script_deploy.sh",
13     "dados_clientes.db"
14 };
15 #define NNOS (int)(sizeof(nomes) / sizeof(nomes[0]))
16
17 static no_t *criar_lista(void) {
18     no_t *head = NULL, *tail = NULL;
19     for (int i = 0; i < NNOS; i++) {
20         no_t *n = malloc(sizeof(no_t));
21         n->nome = nomes[i];
22         n->next = NULL;
23         if (!tail) head = tail = n;
24         else { tail->next = n; tail = n; }
25     }
26     return head;
27 }
28
29 static void liberar_lista(no_t *head) {
30     while (head) { no_t *t = head->next; free(head); head = t; }
31 }
32
33 static void processar(const no_t *n) {
34     printf("[thread %d] %s\n", omp_get_thread_num(), n->nome);
35 }
36
37 /* — Versão 1: problemática —————
38 Todos os threads percorrem a lista independentemente e cada um cria
39 uma tarefa para cada nó.
40 Resultado: NTHREADS x NNOS tarefas → cada arquivo processado NTHREADS vezes. */
41 static void versao_problematICA(no_t *head) {
42     #pragma omp parallel num_threads(NTHREADS)
43     {
44         no_t *p = head;
45         while (p) {
46             #pragma omp task firstprivate(p)
47             processar(p);
48             p = p->next;
49         }
50         #pragma omp taskwait
51     }
52 }
53
54 /* — Versão 2: correta —————
55 #pragma omp single garante que apenas UMA thread percorre a lista e
56 cria as tarefas. As demais ficam disponíveis para executá-las.
57 firstprivate(p): cada tarefa captura o valor corrente de p no momento
58 da criação – sem isso, p seria shared e poderia já ter avançado antes
59 da tarefa executar.
60 O taskwait implícito no final do single aguarda todas as tarefas. */
61 static void versao_correta(no_t *head) {
62     #pragma omp parallel num_threads(NTHREADS)
63     {
64         #pragma omp single
65         {
66             no_t *p = head;
67             while (p) {
68                 #pragma omp task firstprivate(p)
69                 processar(p);
70                 p = p->next;
71             }
72         }
73     }
74 }
75
76 int main(void) {
77     no_t *lista = criar_lista();
78
79     printf("=== Versão problemática (%d threads, %d nós → esperado %d linhas) ===\n",
80           NTHREADS, NNOS, NTHREADS * NNOS);
81     versao_problematICA(lista);
82
83     printf("\n=== Versão correta (%d threads, %d nós → esperado %d linhas) ===\n",
84           NTHREADS, NNOS, NNOS);
85     versao_correta(lista);
86
87     liberar_lista(lista);
88     return 0;
89 }

```